



**SCCRAPPY** is a modern and distinctive name created for a **Self-Healing Web Scraper**. The name combines the idea of “**Scraping**” with a unique and memorable identity, directly representing the primary purpose of the project: automatically extracting useful data from websites. I selected this name because the project is designed to go beyond traditional web scraping. Conventional web scrapers often depend on fixed website structures and can fail when a website changes its HTML layout or data structure. SCCRAPPY is designed to address this challenge by detecting scraping failures and using an intelligent repair mechanism to adapt and retry the extraction process.

### **Responsible Use**

SCCRAPPY is designed for educational, research, and legitimate automation purposes.

Users should always:

- \* Respect website Terms of Service.
- \* Follow applicable laws and regulations.
- \* Respect Privacy conditions where appropriate.
- \* Avoid excessive requests that may affect website performance.
- \* Handle publicly available data responsibly.

Designed By

**Abhishek Ugare**

## Contents

|                                                               |    |
|---------------------------------------------------------------|----|
| 1. Introduction .....                                         | 04 |
| 1.1 Introduction                                              |    |
| 1.1.1 Sub Points                                              |    |
| 1.2 Need of Project                                           |    |
| 2. Literature Surveys .....                                   | 07 |
| 2.1 Literature Survey                                         |    |
| 2.2 Problem Statement                                         |    |
| 2.3 Problem Solution                                          |    |
| 3. Working Models.....                                        | 09 |
| 3.1 Related Work                                              |    |
| 3.2 System Requirements                                       |    |
| 3.2.1 Software Requirements                                   |    |
| 3.2.2 Hardware Requirements                                   |    |
| 3.3 System Design                                             |    |
| 3.3.1 Technical Diagrams                                      |    |
| 4. Technical Content .....                                    | 15 |
| 4.1 Details of front end of the project                       |    |
| 4.2 Details of back end of the project                        |    |
| 4.3 Details of Agentic AI of the project                      |    |
| 4.4 Connection between front end and back end, Entire details |    |
| 5. Implementation .....                                       | 18 |
| 5.1 Implementation Screenshot/ Snaps                          |    |
| 5.2 Output                                                    |    |
| 5.3 System Testing and Test results table                     |    |
| 5.3.1 Results and Discussion                                  |    |

|                                             |    |
|---------------------------------------------|----|
| 6. Conclusion .....                         | 25 |
| 6.1 Applications or Advantages of project   |    |
| 6.2 Limitations or Disadvantages of project |    |
| 6.3 Future Work / Future scope              |    |
| 6.4 Conclusion                              |    |

## Chapter 1. Introduction

### 1.1 Introduction

Web scraping is the process of automatically collecting and extracting useful information from websites. It is widely used for purposes such as data collection, market research, price monitoring, product analysis, content aggregation, and business intelligence. Traditional web scrapers typically rely on predefined rules, HTML structures, CSS selectors, or XPath expressions to locate and extract information from web pages. However, modern websites frequently change their layouts, HTML structures, class names, and page elements. Even a small structural modification can cause a traditional scraper to fail or return incorrect results. Maintaining these scrapers often requires manual analysis and modification of extraction rules, making the process time-consuming and difficult to manage.

SCCRAPPY is a **Self-Healing Web Scraper** developed to address this challenge by creating a more adaptive and automated scraping system. The project combines a user-friendly web interface with an automated n8n workflow to process website URLs, validate requests, fetch website content, prepare HTML data, and extract useful information. A key feature of SCCRAPPY is its self-healing mechanism. When the normal scraping process does not produce the expected results, the system detects the failure and initiates a repair process. The available website content is analyzed, and an AI-assisted workflow attempts to identify an alternative extraction strategy and retry the process. The complete system follows an automated workflow in which a user submits a website URL through the frontend, the request is sent to the n8n workflow, and the website is processed through multiple stages. These stages include URL validation, website fetching, HTML preparation, data extraction, failure detection, and self-healing when required. The final extracted data can then be structured and provided as a downloadable CSV file.

SCCRAPPY demonstrates how workflow automation and AI-assisted decision-making can improve traditional web scraping systems. Instead of relying entirely on fixed extraction rules, the project introduces an adaptive approach that can detect failures and attempt automated recovery, reducing manual intervention and improving the reliability of the scraping process.

#### 1.1.1 Sub Points

The SCCRAPPY project consists of several integrated components that work together to provide an automated and adaptive web scraping system. Each component performs a specific function in receiving the user request, processing website content, extracting data, detecting failures, and attempting recovery.

- **User-Friendly Frontend**  
The frontend serves as the entry point of the system, allowing users to provide a website URL and initiate the scraping process. It also displays the processing status and final results.
- **URL Validation**  
Before processing the request, the system validates the provided website URL to ensure that it is correctly formatted and suitable for scraping.
- **Automated Website Fetching**  
The system retrieves the available content from the target website through the automated workflow.
- **HTML Content Processing**  
The fetched website content is cleaned and prepared before extraction. Unnecessary elements can be removed to make the content more suitable for analysis.
- **Data Extraction**  
The scraper processes the prepared website content and attempts to identify and extract useful structured information.
- **Failure Detection**  
The system checks whether the scraping process has successfully produced meaningful results. If the expected data is unavailable or extraction fails, the workflow identifies the unsuccessful outcome.
- **Self-Healing Mechanism**  
One of the main features of SCCRAPPY is its self-healing capability. When the normal extraction process fails, the system initiates a repair process that analyzes the available content and attempts to generate an improved extraction approach.
- **n8n Workflow Automation**  
n8n acts as the automation engine of the project. It connects and coordinates different stages of the scraping process, including validation, website fetching, content preparation, extraction, failure handling, and response generation.
- **CSV Output Generation**  
After successful extraction, the collected data is structured into a usable format and made available as a downloadable CSV file.

Together, these components create a complete automated scraping workflow that aims to make web data extraction more adaptive, reliable, and efficient than traditional fixed-rule scraping approaches.

## 1.2 Need of Project

Web scraping has become an important method for collecting publicly available information from websites for applications such as market research, price monitoring, product analysis, data collection, and business intelligence. However, traditional web scraping systems often depend on fixed HTML structures, CSS selectors, or XPath expressions to identify and extract information.

Modern websites frequently change their layouts, class names, page structures, and content organization. When these changes occur, predefined extraction rules may no longer work correctly. As a result, the scraper can fail, return incomplete data, or require manual modification. Maintaining multiple scrapers for different websites can therefore become time-consuming and difficult.

Another major challenge is the lack of automated recovery in traditional scraping systems. When a scraping process fails, developers usually need to manually inspect the website, identify the cause of the failure, modify the extraction logic, and run the scraper again. This increases maintenance effort and reduces the efficiency of automated data collection.

SCCRAPPY was developed to address these challenges by providing a more adaptive and automated web scraping workflow. The system validates the user-provided URL, fetches website content, prepares the HTML for processing, and attempts to extract useful information automatically.

The key requirement of the project is to introduce a **self-healing mechanism** into the scraping process. When the normal extraction workflow does not produce meaningful results, SCCRAPPY detects the failure and initiates an AI-assisted repair process. The system analyzes the available website content and attempts an alternative extraction strategy before retrying the scraping process.

By combining a user-friendly frontend, n8n workflow automation, automated failure detection, AI-assisted repair, and CSV output generation, SCCRAPPY aims to reduce manual intervention and improve the reliability and adaptability of web scraping workflows.

## Chapter 2. Literature Survey

### 2.1 Literature Survey

Web scraping has become an important technique for automatically collecting publicly available information from websites. Traditional web scraping systems generally retrieve web pages and extract information using predefined rules such as HTML tags, CSS selectors, XPath expressions, or regular expressions. These approaches are widely used because they can efficiently collect structured information from websites. However, traditional scraping methods have a major limitation: they are highly dependent on the structure of the target website. Modern websites frequently change their HTML layouts, element structures, class names, and content organization. Even small changes can cause predefined extraction rules to fail, resulting in missing or incorrect data. This creates a continuous maintenance requirement for developers.

Several modern scraping solutions use browser automation and workflow automation to improve the reliability of web data extraction. Browser-based automation can handle dynamic and JavaScript-driven websites, while workflow automation platforms help coordinate tasks such as URL validation, website fetching, content processing, data extraction, error handling, and response generation.

Recent developments in Artificial Intelligence have also introduced more intelligent approaches to web scraping. AI models can analyze unstructured website content, understand patterns, and assist in identifying useful information. This makes it possible to move beyond completely fixed extraction rules and develop more adaptive scraping systems. The concept of self-healing systems has become increasingly relevant in modern software development. A self-healing system can detect failures, analyze the available information, and attempt recovery automatically. Applying this concept to web scraping can reduce manual maintenance by allowing a scraper to identify extraction failures and attempt alternative strategies.

SCCRAPPY is developed based on this approach. The project combines web scraping, workflow automation, failure detection, and AI-assisted repair to create an adaptive scraping workflow. When the normal extraction process does not produce meaningful results, the system initiates a self-healing process that analyzes the available content and attempts to recover the extraction process. Thus, SCCRAPPY represents an approach toward making traditional web scraping more automated, adaptive, and resilient by combining structured workflow automation with intelligent failure recovery mechanisms.

### 2.2 Problem Statement

Web scraping is widely used to collect information from websites, but traditional scraping systems often rely on fixed extraction rules such as CSS selectors, XPath expressions, and

predefined HTML structures. These rules work effectively only as long as the structure of the target website remains unchanged.

Modern websites frequently update their layouts, HTML elements, class names, and content structures. When such changes occur, existing scraping rules may fail to locate the required information. This can result in incomplete data, unsuccessful scraping operations, or incorrect outputs.

In traditional scraping systems, failures usually require manual intervention. Developers must inspect the changed website structure, identify the cause of the failure, modify the extraction logic, and run the scraper again. This process becomes difficult and time-consuming, especially when multiple websites or frequently changing web pages are involved. Another challenge is the lack of automated failure recovery. Most conventional scrapers can detect an error but cannot intelligently analyze the available content and adapt their extraction strategy automatically. Therefore, there is a need for a more adaptive web scraping system that can validate user requests, process website content, detect unsuccessful extraction attempts, and automatically initiate a recovery mechanism. The problem addressed by SCCRAPPY is to develop an automated and self-healing web scraping system that reduces dependency on fixed extraction methods and attempts to recover from scraping failures using workflow automation and AI-assisted repair mechanisms.

### 2.3 Problem Solution

SCCRAPPY provides an automated solution to overcome the limitations of traditional web scraping systems. The project combines a user-friendly frontend, n8n workflow automation, website content processing, failure detection, and AI-assisted self-healing mechanisms into a single scraping system.

The process begins when a user submits a website URL through the frontend interface. The URL is sent to the n8n workflow, where it is first validated. After successful validation, the system fetches the available website content and prepares the HTML for further processing. The scraper then attempts to extract useful information from the processed content. The system evaluates the extraction outcome to determine whether meaningful data has been successfully obtained. If the extraction process is unsuccessful, SCCRAPPY does not immediately terminate the workflow. Instead, it activates the **self-healing mechanism**. The available website content and failure information are prepared for analysis, and an AI-assisted repair process attempts to identify an alternative extraction strategy.

The repaired strategy is then used to retry the extraction process. If successful, the extracted information is structured and returned to the user. The final data can be generated in a downloadable CSV format.

Through this approach, SCCRAPPY reduces dependency on completely fixed scraping logic and introduces an adaptive recovery mechanism. The system aims to reduce manual intervention, improve workflow automation, and provide a more resilient approach to web data extraction.



## Chapter 3. Working Models

### 3.1 Related Work

Web scraping has become an important technique for collecting structured information from websites for applications such as price comparison, market analysis, research, monitoring, and data aggregation. Traditional web scrapers generally depend on predefined selectors, XPath expressions, CSS selectors, or fixed page structures. Although these approaches are effective for stable websites, they often fail when a website changes its HTML structure, class names, attributes, or content layout.

Several existing tools and frameworks, such as **Beautiful Soup**, **Scrapy**, **Selenium**, and **Playwright**, provide powerful mechanisms for extracting information from web pages. These technologies can handle different levels of website complexity, including static and dynamically rendered pages. However, most conventional scraping systems require developers to manually modify extraction rules whenever the target website changes.

Cloud-based scraping platforms have also introduced features such as browser automation, proxy management, JavaScript rendering, and anti-bot handling. These services improve the reliability of data collection but generally focus on executing scraping operations rather than automatically identifying and repairing failed extraction logic.

Recent developments in **Artificial Intelligence and Large Language Models (LLMs)** have created opportunities for making web scraping more adaptive. AI-based systems can analyze HTML content, identify patterns in repeated records, understand semantic relationships between fields, and generate extraction rules dynamically. However, many such systems still require human intervention when extraction fails or when the website structure changes significantly.

**SCCRAPPY – Self-Healing Web Scraper** builds upon these existing approaches by introducing an automated recovery mechanism. Instead of simply reporting a scraping failure, SCCRAPPY analyzes the fetched webpage, detects extraction problems, and attempts to generate or repair the extraction logic automatically. The system uses **n8n for workflow orchestration** and AI-assisted processing to create a more adaptive scraping pipeline. This reduces dependence on manually maintained selectors and improves the ability of the scraper to continue functioning when website structures change.

Thus, the proposed system combines the reliability of conventional web-scraping technologies with AI-based reasoning and workflow automation to provide a more **adaptive, fault-tolerant, and self-healing web scraping solution**.

### 3.2 System Requirements

System requirements define the minimum software and hardware specifications necessary for developing, deploying, and running the **SCCRAPPY – Self-Healing Web Scraper** efficiently. Proper system requirements ensure smooth development, reliable workflow execution, stable web scraping operations, and compatibility with different environments.

#### 3.2.1 Software Requirements

The software requirements used during the development of SCCRAPPY are listed below:

| Software/ Technology | Purpose                                            |
|----------------------|----------------------------------------------------|
| HTML5                | Structure of the web-based scraper interface       |
| CSS3                 | Styling, layout, responsiveness, and visual design |
| JavaScript           | Frontend logic, API communication                  |
| n8n                  | Workflow automation                                |

These technologies provide a modern development environment that supports rapid development, reusable components, secure authentication, and efficient database management.

#### 3.2.2 Hardware Requirements

The minimum and basic hardware requirements for developing and running the application are as follows:

| Hardware Component | Minimum Requirement                          |
|--------------------|----------------------------------------------|
| Processor          | Intel Core i3 (4th Generation) or equivalent |
| RAM                | 8 GB                                         |

The application is lightweight and can be accessed through any modern desktop or laptop with an active internet connection.

### 3.3 System Design

System design defines the overall architecture, components, and workflow of the **SCCRAPPY – Self-Healing Web Scraper**. It describes how the frontend, n8n automation workflow, web-scraping services, and AI-based self-healing mechanism interact with each other to provide an automated, reliable, and adaptive web scraping system.

The SCCRAPPY application follows a **workflow-based client-server architecture**. The frontend is developed using **HTML5, CSS3, and JavaScript**, providing a simple and responsive interface through which users can submit website URLs and scraping requests. The frontend communicates with the backend automation system using an **n8n Webhook**.

The backend processing and workflow orchestration are handled by **n8n**. The workflow receives the user's request, validates the URL, fetches the target webpage, processes the returned HTML, and attempts to extract the required information. Different workflow nodes are responsible for individual operations such as URL validation, webpage fetching, HTML preparation, data extraction, result validation, and response generation.

A key feature of SCCRAPPY is its **self-healing mechanism**. When the initial scraping process fails because of changes in webpage structure, missing selectors, invalid extraction logic, or unavailable data, the workflow identifies the failure and passes the relevant webpage content to an **AI/LLM-based repair process**. The AI analyzes the available HTML structure and generates an improved extraction strategy. The repaired logic is then used to retry the extraction process.

External web-scraping infrastructure such as **Bright Data** can be used to fetch webpages reliably, particularly when websites require JavaScript rendering or additional scraping capabilities. The extracted information is processed into a structured format such as JSON and returned to the frontend, where the results, status, or error information are displayed to the user.

The overall workflow begins when a user enters a target URL and submits a scraping request. The request is sent to the n8n webhook, where the URL is validated. If the URL is valid, the target webpage is fetched and its HTML content is prepared for processing. The scraper then attempts to identify and extract the required data. If extraction succeeds, the structured results are returned to the user. If extraction fails, the self-healing mechanism analyzes the webpage and attempts to repair the extraction process before retrying. If the repair is unsuccessful, an appropriate failure response is returned instead of silently producing incorrect data.

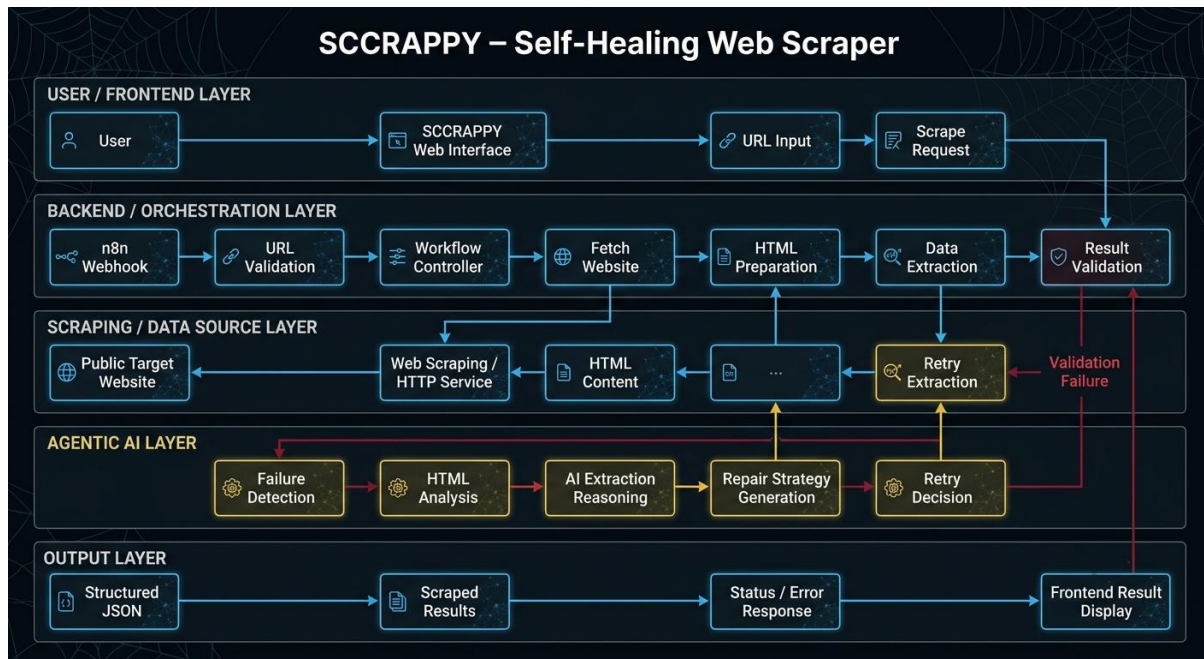
This architecture makes SCCRAPPY more flexible than a conventional fixed-selector scraper because the system can **detect failures, analyze the cause, repair extraction logic, and retry automatically**.

#### 3.3.1 Technical Diagrams

The System Architecture Diagram represents the major components of SCCRAPPY, including the user interface, n8n workflow, web-scraping service, AI repair mechanism, and result-processing layer.

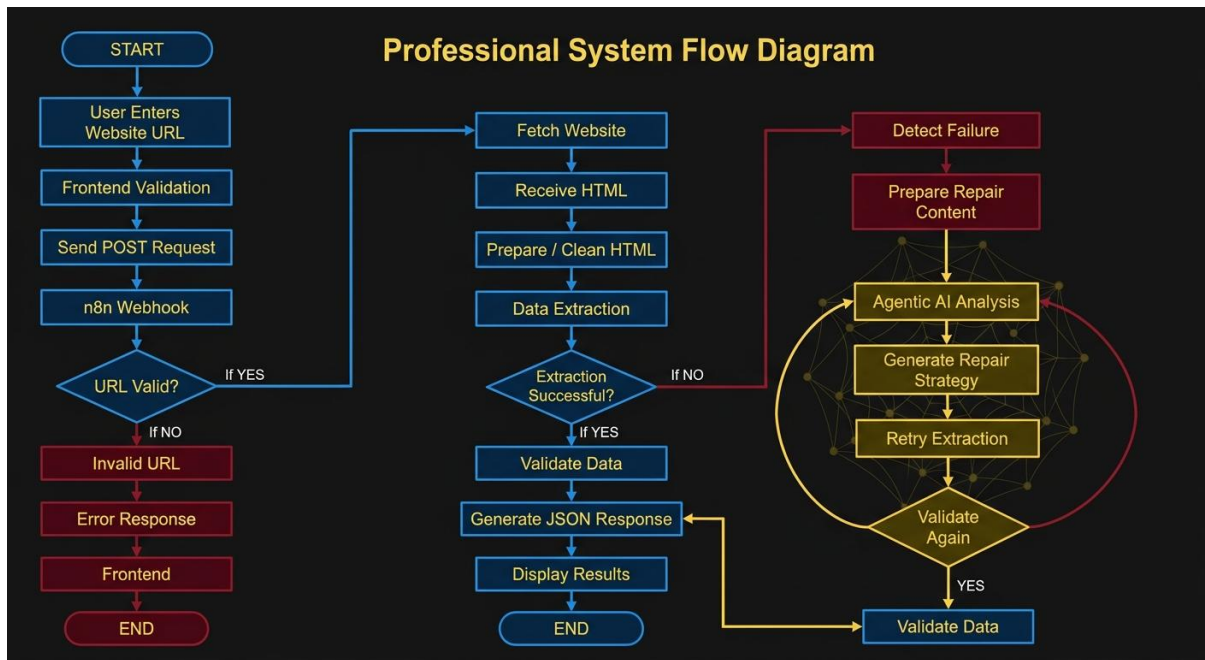
### • System Architecture Diagram

The System Architecture Diagram represents the major components of SCCRAPPY, including the user interface, n8n workflow, web-scraping service, AI repair mechanism, and result-processing layer.



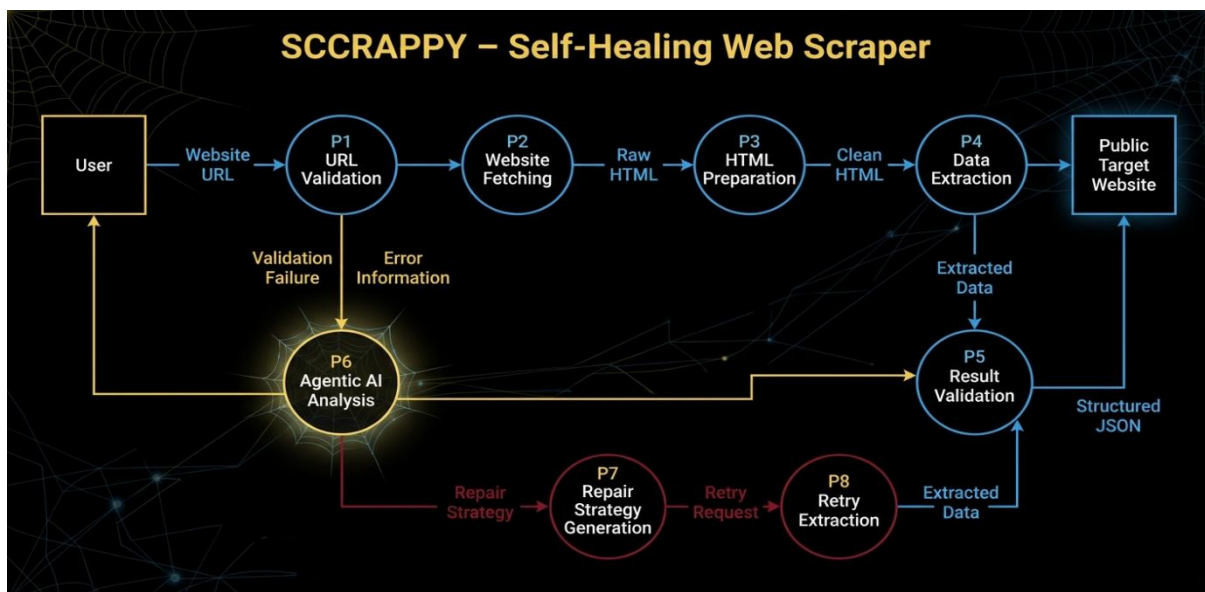
### • System Flow Diagram

The System Flow Diagram represents the complete execution flow from URL submission and validation to webpage fetching, data extraction, error detection, self-healing, and final response generation.



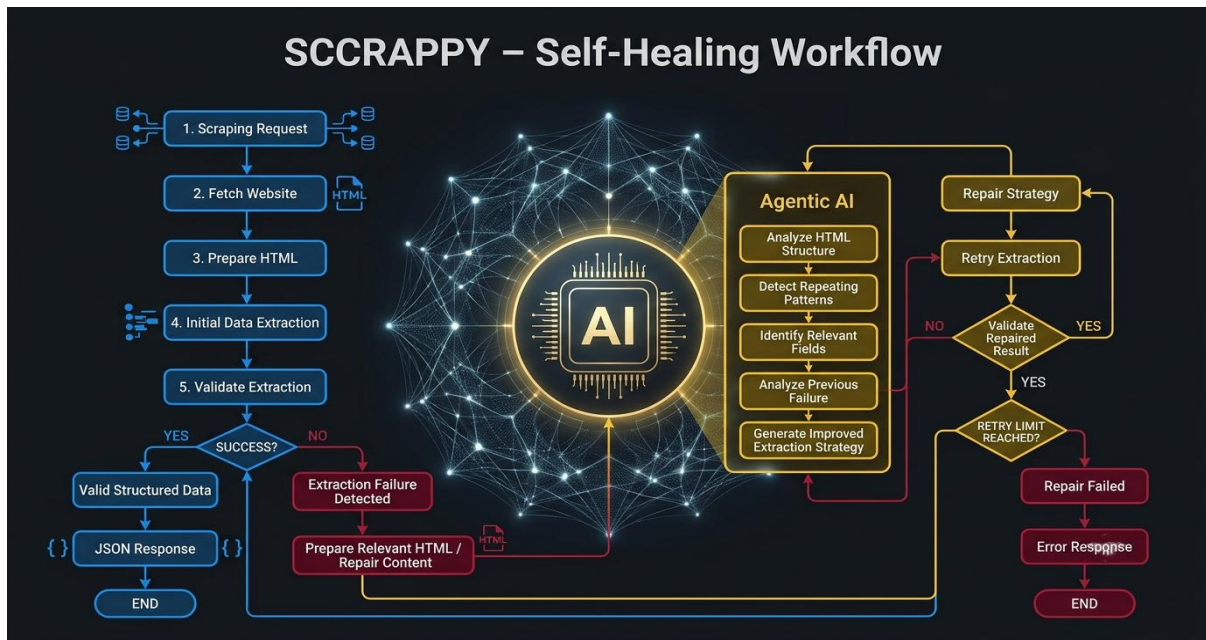
### • Data Flow Diagram

The Data Flow Diagram illustrates how the URL, webpage HTML, extraction instructions, scraped data, error information, and repaired extraction logic move between different components of the system.

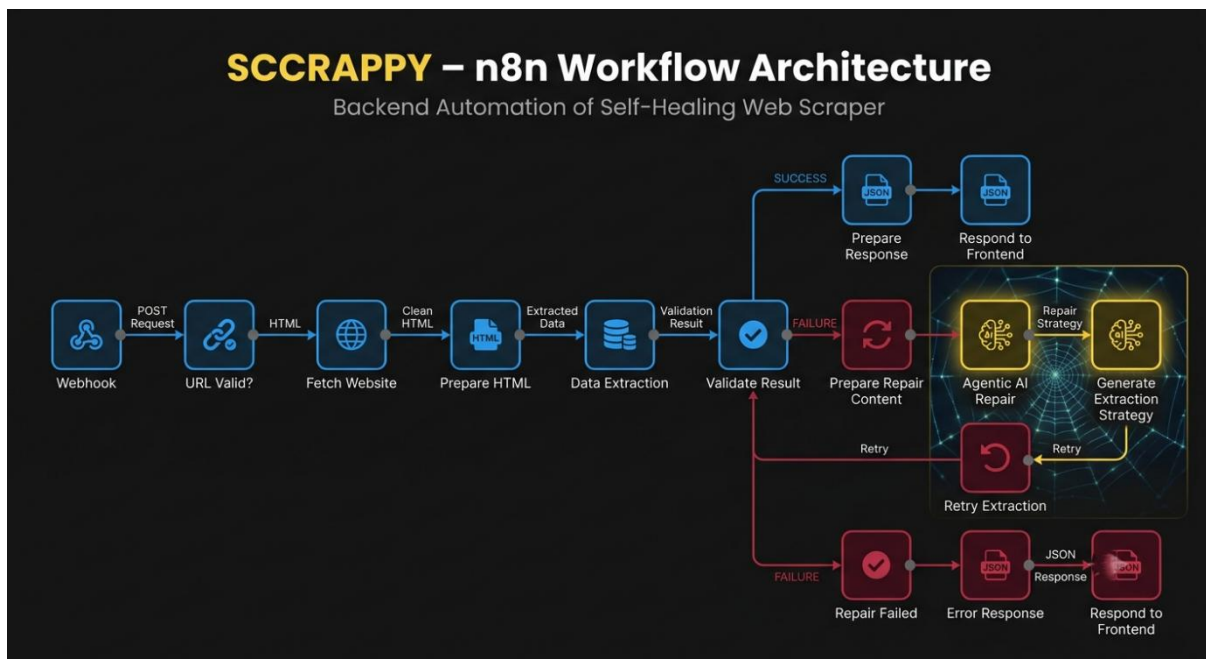


### • Self-Healing Workflow Diagram

The Self-Healing Workflow Diagram explains the core functionality of SCCRAPPY. It shows how the system detects an extraction failure, prepares relevant HTML content, sends it to the AI repair mechanism, receives the repaired extraction strategy, and retries the scraping operation.



#### • n8n Workflow Architecture



## Chapter 4. Technical Content

### 4.1 Frontend Details

The frontend of **SCCRAPPY – Self-Healing Web Scraper** provides the main interaction point between the user and the scraping system. It is designed to provide a simple, modern, responsive, and easy-to-use interface through which users can submit website URLs and monitor the scraping process.

The frontend is developed using **HTML5, CSS3, and JavaScript**. HTML5 is used to create the structure of the application, CSS3 is responsible for the visual design, layout, animations, responsiveness, and user interface styling, while JavaScript manages user interactions, URL validation, communication with the n8n workflow, and displaying the scraping results.

The frontend communicates with the backend automation workflow through an **n8n Webhook** using HTTP POST requests. When the user submits a URL, JavaScript validates the input and sends the request to the configured n8n webhook. The workflow processes the request and returns a structured JSON response containing the scraping status, extracted data, error information, or self-healing results.

The interface also provides different visual states to clearly communicate the current status of the scraping operation. These states may include **processing, successful extraction, self-healing/repair, no data found, service unavailable, and error**. This allows users to understand what is happening without needing to view the underlying n8n workflow.

### Frontend Technologies Used

- HTML
- CSS

## 4.2 Backend Details

The backend of **SCCRAPPY – Self-Healing Web Scraper** is responsible for managing the complete scraping workflow, processing webpage content, detecting extraction failures, and coordinating the self-healing mechanism. Unlike a traditional backend implemented as a single server application, SCCRAPPY uses **n8n as a workflow automation and orchestration layer**.

The frontend communicates with the backend through an **n8n Webhook** using HTTP POST requests. The webhook acts as the entry point of the scraping system and receives the target URL and other required parameters from the frontend. After receiving a request, the n8n workflow performs a sequence of processing operations. First, the submitted URL is validated to ensure that it is properly formatted and accessible for processing. If the URL is valid, the workflow sends a request to the target website through the configured web-fetching or scraping service. The returned HTML content is then prepared by removing unnecessary elements such as scripts, styles, SVG content, comments, and excessive whitespace. This produces cleaner input for the extraction and AI-processing stages. The backend then attempts to identify and extract the required information from the webpage. The extracted data is validated to determine whether meaningful results have been obtained. If valid data is found, the workflow prepares a structured JSON response and sends it back to the frontend.

One of the major components of the backend is the **self-healing mechanism**. When the initial extraction fails because of changes in the website structure, missing selectors, invalid extraction logic, or insufficient data, the workflow enters a repair process. Relevant HTML content and information about the extraction failure are provided to an **AI/LLM-based repair component**. The AI analyzes the webpage structure and attempts to generate an improved extraction strategy.

The repaired extraction logic is then used to retry the scraping process. If the retry successfully produces valid data, the results are returned to the frontend. If the repair process also fails, the backend returns an appropriate error or failure status instead of returning unreliable or empty results.

### Backend Services

- N8N Automation

The backend has been designed with scalability in mind, allowing future integration of AI services and workflow automation without major architectural changes.

## 4.3 Agentic AI Details



The **Agentic AI** component is one of the core features of **SCCRAPPY – Self-Healing Web Scraper**. It provides the system with the ability to analyze scraping failures, reason about webpage structures, generate improved extraction strategies, and retry the scraping process automatically.

Unlike a conventional scraper that depends entirely on fixed CSS selectors or XPath expressions, SCCRAPPY uses an AI-driven approach in which the system can respond to changes in the structure of a webpage. The Agentic AI works together with the **n8n workflow** to monitor the scraping process and determine the appropriate action based on the result.

The Agentic AI process begins after the webpage has been fetched and prepared. The relevant HTML content is provided to the AI along with information about the expected extraction task and, when applicable, details about the previous extraction failure. The AI analyzes the HTML structure and identifies repeated elements, attributes, headings, containers, links, and other patterns that can be used to locate the required information.

If the initial extraction is successful, the workflow proceeds normally. However, if the extracted data is empty, invalid, or does not match the expected structure, the system enters the **self-healing process**. The Agentic AI analyzes the available webpage information and determines a new extraction approach rather than requiring the developer to manually modify the scraper.

#### 4.4 Connection between Front End, Back End, and AI Agent

The **SCCRAPPY – Self-Healing Web Scraper** follows an integrated architecture in which the **Frontend, Backend, and Agentic AI** work together to provide an automated and self-healing web scraping process. Each layer has a specific responsibility while communicating with the other components through structured HTTP requests, workflow execution, and JSON data.

The **Frontend** acts as the user interaction layer. It allows the user to enter a target website URL and initiate a scraping request. Using JavaScript, the frontend validates the input and sends the request to the **n8n Webhook**, which serves as the entry point to the backend workflow.

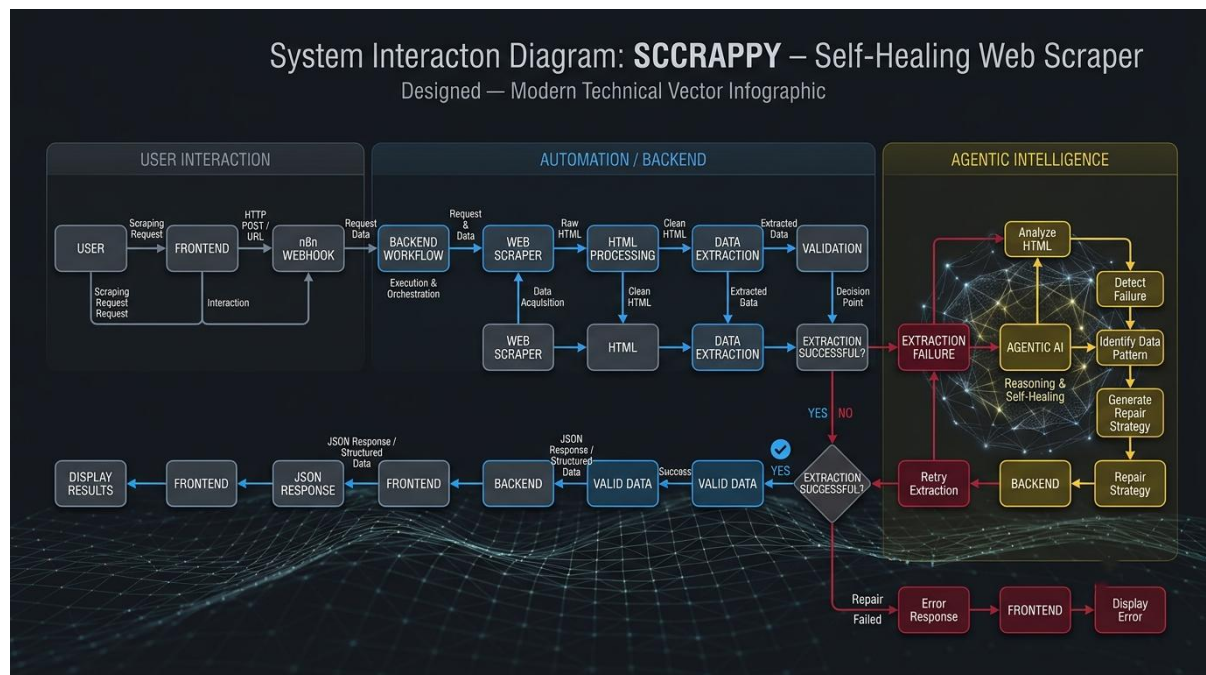
The **Backend**, implemented primarily through **n8n workflow automation**, receives the request and manages the complete scraping process. It validates the URL, fetches the target webpage using the configured scraping or HTTP service, prepares the HTML content, and performs the initial extraction. The backend also validates the extraction result to determine whether meaningful data has been obtained.

The **Agentic AI** acts as the intelligence and recovery layer of the system. When the backend detects that the initial extraction has failed or produced insufficient results, relevant HTML content and failure information are passed to the AI component. The AI analyzes the webpage structure, identifies useful patterns and data fields, and generates an improved extraction strategy. The backend then uses this strategy to retry the extraction process.

After successful extraction, the backend converts the result into a structured JSON response and sends it back to the frontend. The frontend processes the response and displays the extracted information or an appropriate status message to the user.

The planned workflow can be summarized as follows:

## Present Version (Version 1)



## Chapter 5. Implementation

The implementation of **SCCRAPPY – Self-Healing Web Scraper** focuses on converting the proposed system design into a functional web-based scraping application. The system integrates a frontend interface, n8n workflow automation, webpage-fetching services, HTML processing, data extraction, validation, and an Agentic AI-based self-healing mechanism.

The implementation follows a modular approach in which each component performs a specific task. The frontend is responsible for collecting user input and displaying results, while the n8n workflow manages the backend processing and coordinates the scraping and self-healing operations.

### 5.1 Implementation Screenshots / Snaps

The following screenshots demonstrate the successful implementation of the Yaatra application.



Figure 5.1 Home Page

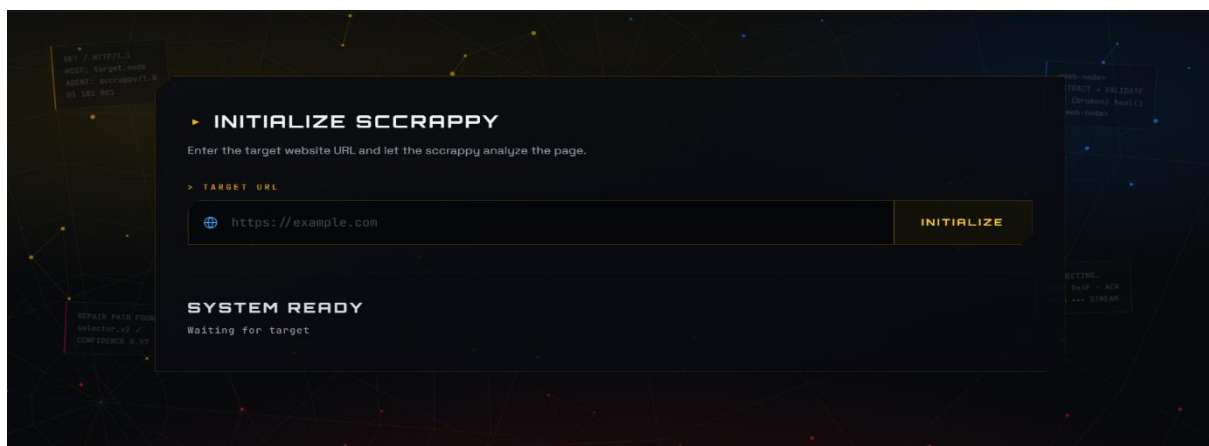


Figure 5.2 Initialization

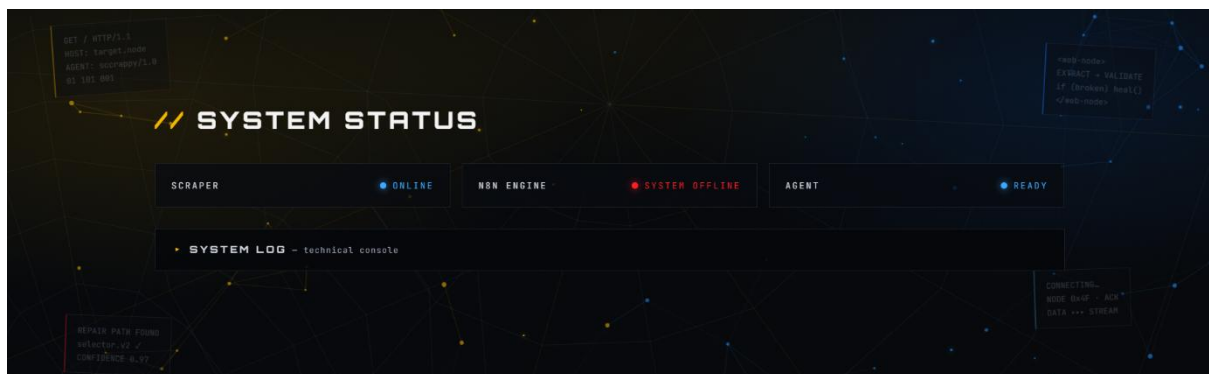


Figure 5.3 System Status

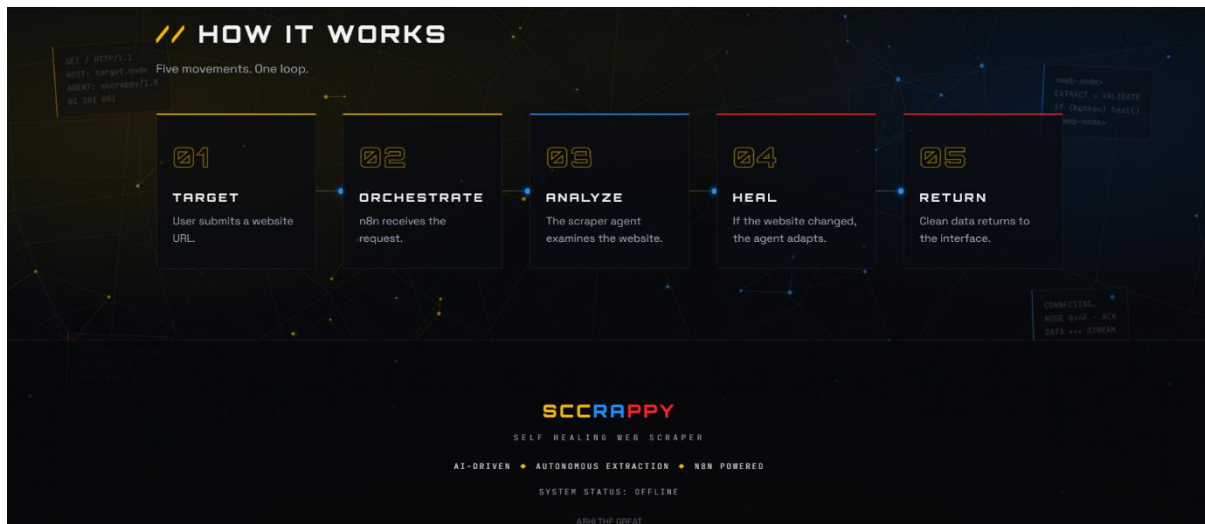


Figure 5.4 Work Mechanism

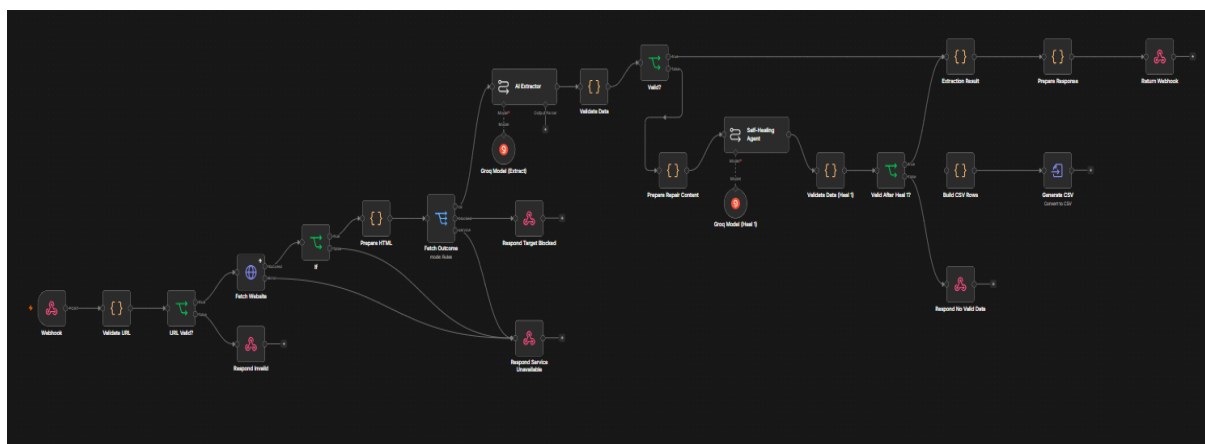


Figure 5.5 Sccrappy Backend Service ( AI Agent )

## 5.2 Output

The **SCCRAPPY – Self-Healing Web Scraper** successfully performs automated web scraping and generates structured data from the target websites provided by the user. After the user enters a valid website URL and submits the scraping request, the frontend sends the request to the n8n workflow, which processes the webpage through the scraping and data extraction pipeline.

The generated output includes the **scraping status, extracted records, relevant data fields, and processing information**. Depending on the target website, the extracted information may include product names, prices, ratings, descriptions, links, or other repeated webpage data. The results are returned in a structured format and displayed through the SCCRAPPY frontend.

One of the major outputs of the system is the **self-healing response**. If the initial extraction fails because of changes in the webpage structure or extraction logic, SCCRAPPY activates the Agentic AI mechanism. The AI analyzes the available HTML content, generates a revised extraction strategy, and allows the workflow to retry the scraping process. When the repair is successful, the system returns the recovered data along with an appropriate status.

The application also provides different output states such as **successful extraction, self-healed extraction, no data found, service unavailable, and error**. These states allow users to clearly understand the result of their scraping request.

The output demonstrates that SCCRAPPY can perform automated data extraction while also attempting to recover from common scraping failures. By combining **web scraping, n8n workflow automation, validation, and Agentic AI-based self-healing**, the system provides a more adaptive and reliable alternative to conventional fixed-rule web scrapers.

### 5.3 System Testing and Test Results Table

The testing process included validation of URL input, webhook communication, webpage fetching, HTML preparation, data extraction, extraction-result validation, AI-based repair, retry mechanism, response generation, error handling, and frontend result display. Both successful and failure scenarios were tested to ensure that the system could properly handle normal scraping operations as well as common scraping failures.

**Test Results Table**

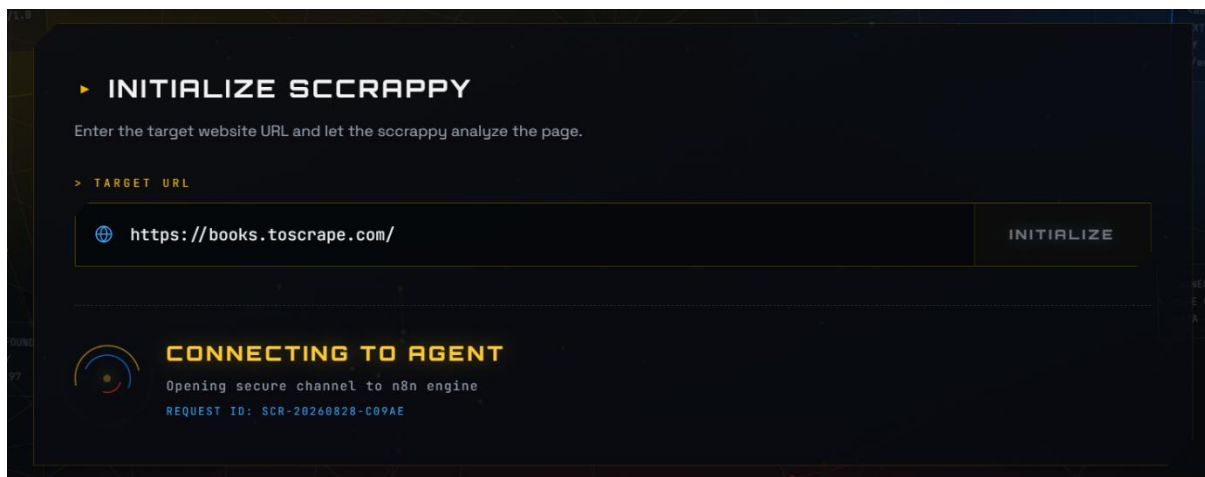
| Module           | Test Description                 | Expected Result                                     | Actual Result                          | Status |
|------------------|----------------------------------|-----------------------------------------------------|----------------------------------------|--------|
| URL Validation   | Submit a valid website URL       | URL should be accepted and workflow should continue | Valid URL accepted successfully        | Pass   |
| URL Validation   | Submit an invalid URL            | Appropriate validation error should be generated    | Invalid URL handled correctly          | Pass   |
| Website Fetching | Fetch a valid webpage            | Target webpage HTML should be retrieved             | Webpage fetched successfully           | Pass   |
| HTML Preparation | Process fetched HTML             | Unnecessary HTML elements should be removed         | HTML cleaned and prepared successfully | Pass   |
| Data Extraction  | Extract information from webpage | Relevant structured records should be generated     | Data extracted successfully            | Pass   |

|                     |                                            |                                                                           |                                         |             |
|---------------------|--------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------|-------------|
| Result Validation   | Validate extracted data                    | System should identify whether meaningful data exists                     | Extraction result validated correctly   | <b>Pass</b> |
| JSON Response       | Return processed result to frontend        | Structured response should be generated                                   | JSON response received successfully     | <b>Pass</b> |
| Responsive Design   | Open application on different screen sizes | Interface should adapt to different screen sizes                          | Layout worked correctly                 | <b>Pass</b> |
| End-to-End Workflow | Execute complete scraping process          | Request should pass through all workflow stages and return a final result | Complete workflow executed successfully | <b>Pass</b> |

### Test Screenshots :

#### Test 1

Input :



Execution :

**EXECUTION RESULT** SCRAPE SUCCESS

|                                                                                 |                          |                                         |
|---------------------------------------------------------------------------------|--------------------------|-----------------------------------------|
| TARGET<br><a href="https://books.toscrape.com/">https://books.toscrape.com/</a> | STATUS<br><b>SUCCESS</b> | ITEMS FOUND<br><b>7</b>                 |
| EXECUTION TIME<br>—                                                             | SELF-HEALED<br><b>NO</b> | REQUEST ID<br><b>SCR-20260829-C86C9</b> |

**SCRAPER HEALTH**  
● **STABLE**  
 Extraction strategy matched the current page structure.

Result :

**EXTRACTED DATA** TABLE VIEW JSON VIEW DOWNLOAD CSV

| NAME                                               | PRICE  | RATING | URL                                                                         | AVAILABILITY |
|----------------------------------------------------|--------|--------|-----------------------------------------------------------------------------|--------------|
| A Light in the Attic                               | £51.77 | Three  | catalogue/a-light-in-the-attic_1000/index.html                              | In stock     |
| Tipping the Velvet                                 | £53.74 | One    | catalogue/tipping-the-velvet_999/index.html                                 | In stock     |
| Soumission                                         | £50.10 | One    | catalogue/soumission_998/index.html                                         | In stock     |
| Sharp Objects                                      | £47.82 | Four   | catalogue/sharp-objects_997/index.html                                      | In stock     |
| Sapiens: A Brief History of Humankind              | £54.23 | Five   | catalogue/sapiens-a-brief-history-of-humankind_996/index.html               | In stock     |
| The Requiem Red                                    | £22.65 | One    | catalogue/the-requiem-red_995/index.html                                    | In stock     |
| The Dirty Little Secrets of Getting Your Dream Job | £33.34 | Four   | catalogue/the-dirty-little-secrets-of-getting-your-dream-job_994/index.html | In stock     |

## Test 2

Input :

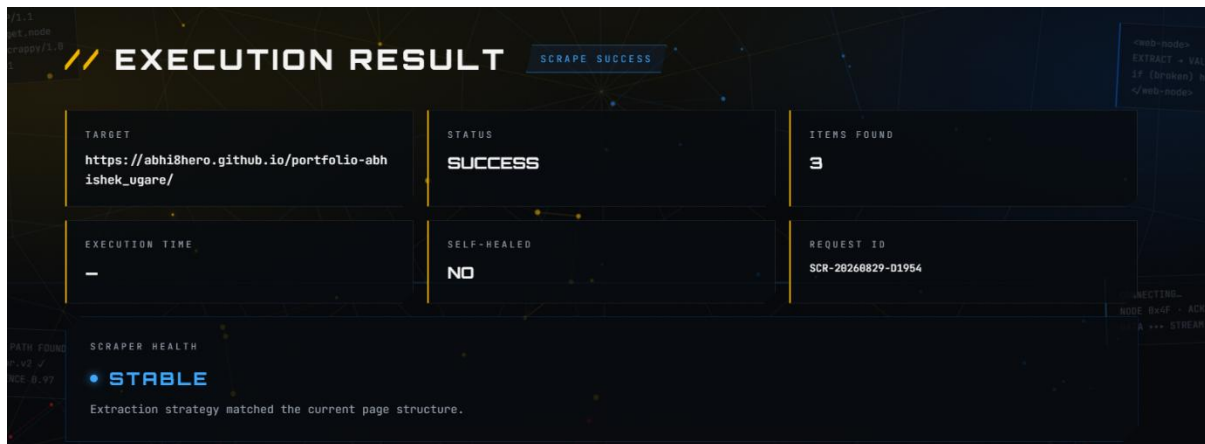
**INITIALIZE SCCRAPPY**  
 Enter the target website URL and let the sccrappy analyze the page.

> **TARGET URL**

[https://abhi8hero.github.io/portfolio-abhishek\\_ugare/](https://abhi8hero.github.io/portfolio-abhishek_ugare/) INITIALIZE

**SCRAPE COMPLETE**  
 Clean data returned to the interface  
 REQUEST ID: SCR-20260829-D1954

Execution :



Result :

| EXTRACTED DATA                            |                                                                                    |                        |         |
|-------------------------------------------|------------------------------------------------------------------------------------|------------------------|---------|
|                                           |                                                                                    |                        |         |
| DEGREE                                    | INSTITUTION                                                                        | PERIOD                 | GRADE   |
| B.Tech - Computer Science and Engineering | Shri Balasaheb Mane Shikshan Prasarak Mandal's Ashokrao Mane Group Of Institutions | 2023 - 2027 (pursuing) |         |
| 12th HSC Board - Science                  | Annasaheb Dange Public School and Junior College, Ashta                            | 2021 - 2023            | 61.83 % |
| 10th SSC Board                            | Vilasrao Shinde Secondary and Higher Secondary School, Ashta                       | 2021                   | 80.40 % |

### 5.3.1 Results and Discussion

The implementation and testing of the **SCCRAPPY – Self-Healing Web Scraper** demonstrate that the system successfully achieves the objectives defined during the project planning phase. The major components, including URL validation, webpage fetching, HTML preprocessing, data extraction, result validation, error handling, and AI-assisted self-healing, function as intended.

The frontend communicates efficiently with the **n8n backend workflow** through the webhook interface. The workflow successfully receives scraping requests, processes the target webpage, and returns structured responses to the frontend. The implementation of separate workflow stages makes the scraping process organized, manageable, and easier to debug.

The **Agentic AI integration** is the key feature of SCCRAPPY. When the initial extraction fails because of changes or inconsistencies in the webpage structure, the system can analyze the available HTML content and generate an alternative extraction strategy. The retry mechanism then attempts to extract the required information using the repaired logic. This demonstrates the system's ability to recover from certain scraping failures without requiring immediate manual modification.



Testing also showed that the system can handle different execution states, including successful extraction, no-data situations, invalid URLs, webpage or service failures, and self-healing scenarios. The frontend provides appropriate status information so that users can understand whether the scraping operation was successful, required repair, or encountered an error.

Overall, the testing results confirm that SCCRAPPY provides a **functional, adaptive, and fault-tolerant web scraping solution**. By combining conventional web scraping with **n8n workflow automation and Agentic AI-based self-healing**, the system reduces dependence on fixed extraction rules and improves its ability to handle changes in webpage structures. The modular architecture also provides opportunities for future enhancements such as improved AI reasoning, support for more complex websites, advanced extraction techniques, monitoring, and automated scraper optimization.

## Chapter 6. Conclusion

### 6.1 Applications / Advantages of the Project

**SCCRAPPY – Self-Healing Web Scraper** is an intelligent web-based scraping system designed to automate the process of collecting structured information from websites. The system combines conventional web scraping techniques with **n8n workflow automation and Agentic AI** to improve the reliability and adaptability of data extraction.

Unlike traditional scrapers that depend heavily on fixed selectors and require manual updates when webpage structures change, SCCRAPPY attempts to detect extraction failures and automatically generate a repair strategy. This makes the system useful for applications where websites frequently change their structure or where continuous data collection is required.

The major applications and advantages of the project are as follows:

- **Automated Web Data Extraction:**  
SCCRAPPY automatically retrieves and extracts relevant information from target websites, reducing the need for repetitive manual data collection.
- **Self-Healing Scraping:**  
When the initial extraction fails because of changes in webpage structure, the Agentic AI analyzes the available HTML and attempts to generate an improved extraction strategy.
- **Reduced Manual Maintenance:**  
Traditional scrapers often require developers to manually update selectors whenever a website changes. SCCRAPPY reduces this maintenance effort by providing an automated repair mechanism.
- **Workflow Automation:**  
n8n coordinates URL validation, webpage fetching, HTML processing, extraction, validation, AI repair, retry, and response generation within a single workflow.
- **Dynamic Data Extraction:**  
The system can identify repeated webpage structures and extract relevant records such as product names, prices, ratings, descriptions, links, and other available information.
- **Error Detection and Recovery:**  
SCCRAPPY validates extraction results and can distinguish between successful extraction, no-data conditions, service failures, and extraction errors.

## 6.2 Limitations / Disadvantages of the Project

Although the current version of **SCCRAPPY – Self-Healing Web Scraper** successfully fulfills its primary objectives, certain limitations exist due to the complexity, variability, and security mechanisms of modern websites. These limitations can be addressed and improved in future versions of the system.

Some of the current limitations are:

- **Internet Dependency:** The application requires an active internet connection because it needs to communicate with target websites, scraping services, n8n, and AI services.
- **Protected Websites:** **SCCRAPPY must not be used to scrape protected, restricted, private, or access-controlled websites**, including websites that prohibit automated access. The system is intended for publicly accessible web content where automated collection is permitted.
- **Robots.txt and Website Policies:** Scraping must respect the target website's robots.txt, terms of service, copyright requirements, rate limits, and other applicable policies. SCCRAPPY does not override website access restrictions.
- **Anti-Bot and CAPTCHA Protection:** Websites using advanced anti-bot systems, CAPTCHAs, authentication requirements, or sophisticated access controls may prevent successful scraping.
- **Dynamic Websites:** Highly dynamic websites that rely heavily on JavaScript, client-side rendering, infinite scrolling, or interactive components may require additional browser automation or specialized handling.
- **AI Accuracy:** The self-healing mechanism depends on the quality of the HTML content and the AI model's ability to correctly understand the webpage structure. The generated repair strategy may not always produce the desired result.
- **Large Webpages:** Very large or complex HTML documents may require preprocessing or truncation before being passed to the AI system, which can potentially exclude useful information.
- **Extraction Reliability:** The system may not correctly identify the required data when webpages have inconsistent structures, poorly organized HTML, or ambiguous content.
- **External Service Dependency:** The performance of the system may depend on external services such as web-fetching/scraping infrastructure and AI model availability.
- **Rate Limits and Availability:** Target websites or external services may impose request limits, temporary restrictions, or service interruptions that can affect scraping operations.

- **No Guaranteed Self-Healing:** The AI-based repair mechanism improves recovery from certain extraction failures, but it cannot guarantee successful recovery from every possible website structure or access restriction.
- **Processing Cost:** Repeated AI analysis and retry operations may increase processing time and resource consumption compared with a simple fixed-rule scraper.
- **Security and Ethical Considerations:** Users must ensure that collected information is obtained legally and ethically. The system should not be used to bypass authentication, access controls, paywalls, CAPTCHAs, or other technical restrictions.

### 6.3 Future Work / Future Scope

The current version of **SCCRAPPY – Self-Healing Web Scraper** provides an effective foundation for automated and adaptive web data extraction. However, the system can be further enhanced with advanced AI capabilities, improved scraping techniques, stronger monitoring, and greater scalability.

The possible future improvements and scope of SCCRAPPY are as follows:

- **Advanced Multi-Agent Architecture:** Future versions can use multiple specialized AI agents for webpage analysis, extraction, validation, error diagnosis, and repair, allowing different agents to collaborate on complex scraping tasks.
- **Improved Self-Healing Mechanism:** The AI repair system can be enhanced to learn from previous extraction failures and successful repairs, improving its ability to handle recurring webpage structure changes.
- **Automatic Selector Generation:** The system can automatically generate and compare CSS selectors, XPath expressions, and semantic extraction rules to identify the most reliable extraction strategy.
- **Dynamic Website Support:** Integration with browser automation technologies such as Playwright or Selenium can improve support for JavaScript-heavy and dynamically rendered websites, where permitted.
- **Advanced Data Validation:** Future versions can introduce stronger validation using schemas, field-type detection, duplicate detection, confidence scores, and consistency checks to improve extraction accuracy.
- **Scraper Health Monitoring:** A monitoring dashboard can be developed to track workflow executions, extraction success rates, failures, repair attempts, response times, and scraper health over time.
- **Learning-Based Repair:** Historical scraping results can be used to identify common website changes and improve future repair decisions through machine-learning or feedback-based approaches.

- **Scheduled Scraping:** The system can support scheduled scraping jobs for permitted public websites, allowing users to automatically collect updated information at predefined intervals.
- **Data Export and Integration:** Extracted data could be exported to formats such as CSV and JSON or integrated with databases, APIs, dashboards, and other business applications.
- **Scalable Cloud Deployment:** SCCRAPPY can be deployed using cloud infrastructure and containerization technologies to support larger workloads and multiple scraping workflows.
- **Improved Error Classification:** Future versions can classify failures into categories such as network failure, HTML structure change, missing data, rendering failure, rate limiting, or service unavailability, allowing more targeted recovery strategies.
- **Human-in-the-Loop Review:** For uncertain AI-generated extraction strategies, the system could request human approval before applying a repair, improving reliability for important data-collection tasks.
- **Extraction Confidence Score:** The AI could provide a confidence score for each extraction or repair attempt, helping users identify results that may require verification.
- **Responsible Scraping Controls:** Future versions can include stronger safeguards for respecting robots.txt, website terms, rate limits, access permissions, and other applicable restrictions. The system should continue to avoid protected or access-controlled websites and should not attempt to bypass CAPTCHAs, authentication, paywalls, or other technical protections.
- **Natural Language Scraping Requests:** Users could describe what they want in natural language, such as *“Extract product names and prices from this page,”* and the AI could automatically determine the appropriate fields and extraction strategy.
- **Reusable Scraper Templates:** The system could generate reusable extraction configurations for permitted websites, reducing setup time for recurring scraping tasks.

## 6.4 Conclusion

The **SCCRAPPY – Self-Healing Web Scraper** successfully demonstrates the development of an intelligent and adaptive web scraping system that combines conventional web scraping techniques with **workflow automation and Agentic AI**. The primary objective of the project was to reduce the dependency on fixed extraction rules and provide a mechanism that can identify and recover from common scraping failures.

The system provides a complete scraping workflow starting from **URL validation and webpage fetching to HTML preprocessing, data extraction, result validation, and response generation**. The integration of **n8n** enables these operations to be organized into an

automated workflow, while the frontend provides users with a simple interface for submitting scraping requests and viewing the results.

The most significant feature of SCCRAPPY is its **self-healing mechanism**. When the initial extraction fails due to changes in webpage structure or extraction logic, the Agentic AI analyzes the available webpage content and attempts to generate an improved extraction strategy. The workflow can then retry the extraction process and validate the recovered results. This approach makes the system more adaptive than conventional scrapers that require manual modification whenever a website changes.

System testing demonstrated that the major components of the application function according to their intended requirements. URL validation, webpage fetching, HTML processing, data extraction, error handling, AI-assisted repair, retry operations, and frontend response handling were successfully tested.

However, SCCRAPPY should be used responsibly and only for **permitted, publicly accessible web content**. It must not be used to bypass authentication, CAPTCHAs, paywalls, access controls, or other website protections, and scraping activities should comply with applicable laws, website terms, robots.txt, and rate limits.

Overall, SCCRAPPY provides a strong foundation for **automated, adaptive, and fault-tolerant web scraping**. The project demonstrates how Agentic AI can be integrated with workflow automation to make data extraction more resilient to changes in webpage structure. With future improvements such as multi-agent collaboration, advanced browser automation, intelligent monitoring, improved validation, and learning-based repair, SCCRAPPY has significant potential to evolve into a scalable and intelligent web data extraction platform.



Thank you.

Abhishek Ugare

Email: [abhishekugare1289@gmail.com](mailto:abhishekugare1289@gmail.com)

LinkedIn: [www.linkedin.com/in/abhishek-ugare-a289s85k](https://www.linkedin.com/in/abhishek-ugare-a289s85k)

Portfolio: [https://abhi8hero.github.io/portfolio-abhishek\\_ugare/](https://abhi8hero.github.io/portfolio-abhishek_ugare/)

Github: <https://github.com/abhi8hero>

Repo : <https://github.com/abhi8hero/sccrappy>

Live on : <https://sccrappy.vercel.app/>